

Modular Edit Lenses

Martin Hofmann

LMU Munich, Germany

Abstract. This article is a reading guide to the theory of symmetric edit lenses by Pierce, Wagner, and the author, which form a general framework for the modular construction of bidirectional synchronizers and which generalize the popular lenses framework by Foster and Pierce to a truly symmetric, bidirectional setting.

The article describes both the state-based and the edit-based version, as well as an extended example instantiation involving tree-structured data. The main focus is on edit lenses and the categorical combinators which allow for their modular construction. The article is based on three original research papers [9, 10, 21, 11] and summarises these in a concise form but does not contain new scientific material.

1 Introduction

This article is based on a series of three lectures given by the author about his joint work with Benjamin Pierce and David Wagner on the topic of bidirectional synchronisation based on a symmetric version of lenses [7]. In addition to the slides that are still available on the school website this article fills in most of the oral explanations offered. For more detail the reader should consult the papers [9, 10, 21, 11] on which the lectures were based. This article does not contain any original material beyond those.

Bidirectional synchronisation between data given in two different representations is a ubiquitous task. Typical examples include different file systems, data on the web or in the cloud, different software models, different data formats.

In each case the first step is to specify the translation or more general correspondence between the two representations to be synchronised. Mutually inverse back-and-forth translations between the two representations, when available, are always the best option, but in many cases are not possible because each representation may contain additional data that is not reflected in the other representation.

In the classical framework of (asymmetric) lenses [7] one considers that one representation (the “source”) contains more information than the other (the “view”). Thus, a lens from source set S to view set V comprises two functions

$$\begin{aligned} get &\in S \rightarrow V \\ put &\in V \times S \rightarrow S \end{aligned}$$

subject to the requirements that

$$\begin{aligned} get(put(v, s)) &= s \\ put(get(s), s) &= s \end{aligned}$$

Thus, $get(s)$ extracts the V -view from $s \in S$ and $put(v, s)$ is the result of modifying the V -part of $s \in S$ with a new view $v \in V$ but leaving as they were the additional parts of s that are not reflected by the V -view.

Much research on those lenses has been devoted to the comfortable construction of useful lenses. Rather than writing get and put from scratch and establishing the two equational laws one can now define them from basic building blocks with combinators [7] and also derive a suitable put from a given get [20]. The other direction where put must be provided and get is derived and the laws are checked has also been explored [15] (see also this volume) and recently a reconciliation between the combinatory approach from [7] and the programming style with variables implicit in [19, 20] has been achieved using category-theoretic ideas [16].

In the work described here, **the asymmetry between source and few implicit in lenses is removed**; a *symmetric lens* between two sets X and Y is defined as a stateful back- and forth transformation: it comprises a set C of states, the *complement*, and functions

$$\begin{aligned} putr &\in X \times C \rightarrow Y \times C \\ putl &\in Y \times C \rightarrow X \times C \end{aligned}$$

subject to

$$\frac{putr(x, c) = (y, c')}{putl(y, c') = (x, c)} \quad \text{(PUTRL)}$$

$$\frac{putl(y, c) = (x, c')}{putr(x, c') = (y, c)} \quad \text{(PUTLR)}$$

In addition, C must contain a distinguished element $init \in C$.

Thus, to synchronise between repositories X and Y , one uses a shared state with values in C and initialised with $init$. In order to transport a new value $x \in X$ across the lens one invokes $putr(x, c)$ with c being the current state, initially $init$, which yields a pair (y, c') comprising the corresponding Y -value (y) and the new shared state c' . The shared state may be used to represent components of either X or Y that are not reflected in the other repository.

The article **elaborate** this idea as follows. The next section presents the theory of symmetric lenses as given in [9]. We show how they can be composed both sequentially and in parallel, how they relate to asymmetric lenses, and how to design lens combinators for recursive data structures. Section 3 then describes an extension of the lens framework with edit operations. Rather than transporting through the lens the complete new state of the repository one only transmits the

changes that have occurred since the last synchronisation point. Besides possibly saving bandwidth in a distributed scenario this allows for more precise translations if instead of merely propagating the changes themselves we propagate the edit operation that **lead** to them. This allows us, for example, to distinguish a swap between two entries from erasure followed by insertions of new entries. The allowed editing operations on some data structure or repository thus form an *edit language* and bidirectional synchronisations translate between them. Again, we describe the category-theoretic properties of these *edit lenses*, i.e., the ways in which they can be composed, and also combinators for the construction of lenses to operate on more complex data structures. It turns out that the view of such data structures as recursive types does not lend itself to an extension with edits. Instead, we show how to define meaningful edit lenses on so-called *containers* [1] which are closely related to the combinatorial species from [14].

Section 4 describes an application of edit lenses or rather edit languages to XML processing using the automata-theoretic framework of [4, 8]. We define a useful set of edits for XML trees and demonstrate how these can be typechecked against document type specifications represented by automata. It is based on the article [11].

We conclude with some suggestions for further research and projects.

2 Symmetric Lenses

We begin by introducing some notation for the symmetric lenses that have already been presented in the introduction.

Definition 1 (Symmetric lens). *A lens ℓ from X to Y (written $\ell \in X \leftrightarrow Y$) has three parts: a set of complements C , a distinguished element $\text{init} \in C$, and two functions*

$$\begin{aligned} \text{putr} &\in X \times C \rightarrow Y \times C \\ \text{putl} &\in Y \times C \rightarrow X \times C \end{aligned}$$

satisfying the following laws:

$$\frac{\text{putr}(x, c) = (y, c')}{\text{putl}(y, c') = (x, c)} \quad (\text{PUTRL})$$

$$\frac{\text{putl}(y, c) = (x, c')}{\text{putr}(x, c') = (y, c)} \quad (\text{PUTLR})$$

We use record notation writing $\ell.C$ for the complement set of ℓ , etc.

Figure 1 taken from [9] illustrates the operation of a symmetric lens in a concrete example.

An important feature of symmetric lenses is that they can be sequentially composed as follows.

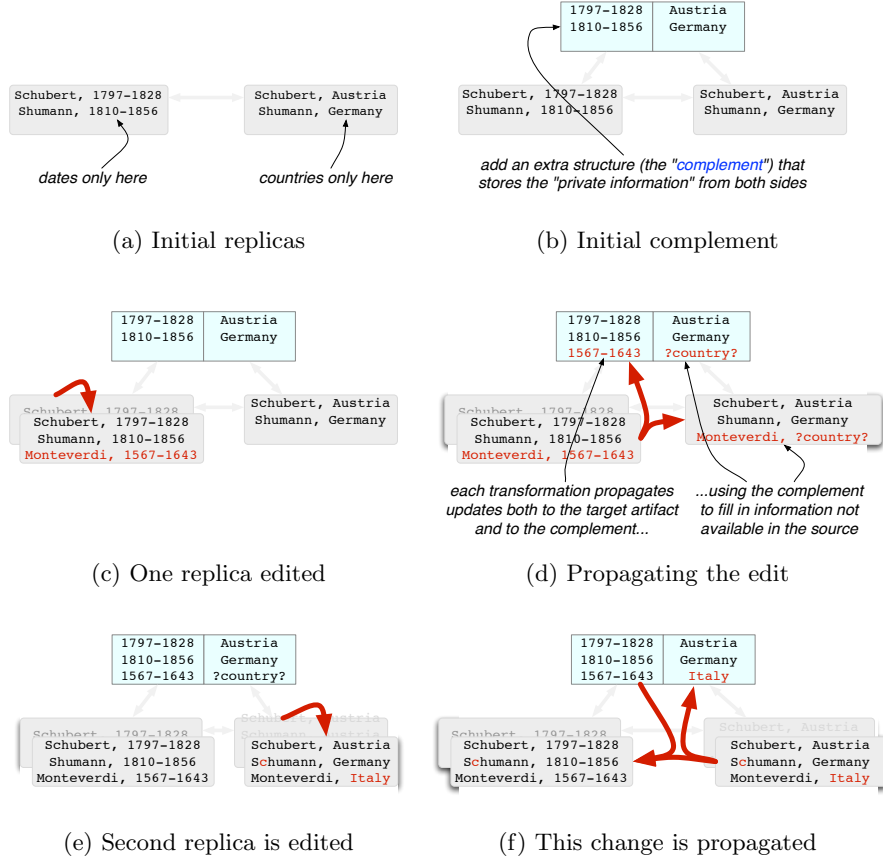


Fig. 1. Behavior of a symmetric lens

Definition 2 (Lens composition).

$$\frac{k \in X \leftrightarrow Y \quad \ell \in Y \leftrightarrow Z}{k; \ell \in X \leftrightarrow Z}$$

$$\begin{aligned} C &= k.C \times \ell.C \\ \text{init} &= (k.\text{init}, \ell.\text{init}) \\ \text{putr}(x, (c_k, c_\ell)) &= \text{let } (y, c'_k) = k.\text{putr}(x, c_k) \text{ in} \\ &\quad \text{let } (z, c'_\ell) = \ell.\text{putr}(y, c_\ell) \text{ in} \\ &\quad (z, (c'_k, c'_\ell)) \\ \text{putl}(z, (c_k, c_\ell)) &= \text{let } (y, c'_\ell) = \ell.\text{putl}(z, c_\ell) \text{ in} \\ &\quad \text{let } (x, c'_k) = k.\text{putl}(y, c_k) \text{ in} \\ &\quad (x, (c'_k, c'_\ell)) \end{aligned}$$

Viewing a symmetric lens as a pair of stateful back-and-forth functions this composition amounts to ordinary composition of functions.

It is important to note that earlier attempts at formalising symmetric bidirectional synchronisation had difficulties with composition.

Consider Meertens' constraint maintainers [17] which were later followed up in  context of model-driven design by Stevens [18], Diskin [5], and Xiong, et al [22]. A constraint maintainer between two sets X and Y has a *consistency relation* $R \subseteq X \times Y$ and two *consistency maintainers* $\triangleleft : X \times Y \rightarrow X$ and $\triangleright : X \times Y \rightarrow Y$ such that $(x \triangleleft y) R y$ and $x R (x \triangleright y)$ always hold, and such that $x R y$ implies $x \triangleleft y = x$ and $x \triangleright y = y$. Now, given such consistency relations $R \subseteq X \times Y$ and $R' \subseteq Y \times Z$ maintained by $\triangleright, \triangleleft$ and $\triangleright', \triangleleft'$, then to construct a maintainer for the composition $R; R'$, we need to, given $x \in X$ and $z \in Z$, come up with a $y \in Y$ that will allow us to use either of the existing maintainer functions, but there is no canonical way of doing that and indeed Meertens gives a concrete counterexample.

2.1 Composition and equivalence

In order to render lens composition associative and also to iron out other unimportant detail it is useful to regard lenses up to behavioural equivalence:

Definition 3. *Two lenses $k, \ell \in X \leftrightarrow Y$ are equivalent, written $k \equiv \ell$, when there is a relation $R \subset k.S \times \ell.S$ and:*

$$\frac{\begin{array}{c} s_k R s_\ell \\ k.putr(a, s_k) = (b_k, s'_k) \\ \ell.putr(a, s_\ell) = (b_\ell, s'_\ell) \end{array}}{b_k = b_\ell \wedge s'_k R s'_\ell}$$

$$\frac{\begin{array}{c} s_k R s_\ell \\ k.putl(b, s_k) = (a_k, s'_k) \\ \ell.putl(b, s_\ell) = (a_\ell, s'_\ell) \end{array}}{a_k = a_\ell \wedge s'_k R s'_\ell}$$

$$\ell.init R k.init$$

It is not hard to see that equivalence of lenses is indeed an equivalence relation and that composition is associative up to $\equiv$. Thus, sets with equivalence classes of symmetric lenses as morphisms form a category.

There is an alternative characterisation of lens equivalence using observations: Given a lens $\ell \in X \leftrightarrow Y$, define a *put object* for ℓ to be a member of $X + Y$. Define a function *apply* taking a lens, an element of that lens' complement set, and a list of put objects, by pushing the list's elements through the lens beginning with the given element. Now, lenses $k, \ell \in X \leftrightarrow Y$ are *observationally equivalent* (written $k \approx \ell$) if, for every sequence of put objects $P \in (X + Y)^*$ we have

$$apply(k, k.init, P) = apply(\ell, \ell.init, P).$$

One can then show that $k \approx \ell$ if and only if $k \equiv \ell$.

2.2 Useful Lenses

We now give a few instance of lenses that keep occurring in examples.

Every bijective function gives rise to a lens:

$$\frac{f \in X \rightarrow Y \quad f \text{ bijective}}{iso_f \in X \leftrightarrow Y}$$

$$\begin{aligned} C &= Unit \\ init &= () \\ putr(x, ()) &= (f(x), ()) \\ putl(y, ()) &= (f^{-1}(y), ()) \end{aligned}$$

For each set X we have the terminal lens

$$\frac{x \in X}{term_x \in X \leftrightarrow Unit}$$

$$\begin{aligned} C &= X \\ init &= x \\ putr(x', c) &= ((), x') \\ putl((), c) &= (c, c) \end{aligned}$$

Being symmetric, lenses can be reversed:

$$\frac{\ell \in X \leftrightarrow Y}{\ell^{op} \in Y \leftrightarrow X}$$

$$\begin{aligned} C &= \ell.C \\ init &= \ell.init \\ putr(y, c) &= \ell.putl(y, c) \\ putl(x, c) &= \ell.putr(x, c) \end{aligned}$$

The disconnect lens allows one to hide information from being transmitted and to store it in the complement.

$$\frac{x \in X \quad y \in Y}{disconnect_{xy} \in X \leftrightarrow Y}$$

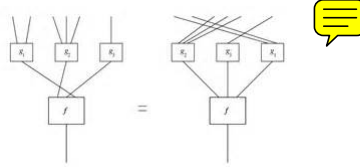
$$disconnect_{xy} = term_x; term_y^{op}$$

We also notice that asymmetric lenses give rise to symmetric lenses and thus can be seen as a special case of the latter.

Besides the sequential composition symmetric lenses can also be juxtaposed thus endowing the category of lenses with a *monoidal structure*.

$\frac{k \in X \leftrightarrow Z \quad \ell \in Y \leftrightarrow W}{k \otimes \ell \in X \times Y \leftrightarrow Z \times W}$	
C $init$ $putr((x, y), (c_k, c_\ell))$ $putl((z, w), (c_k, c_\ell))$	$= k.C \times \ell.C$ $= (k.init, \ell.init)$ $= \text{let } (z, c'_k) = k.putr(x, c_k) \text{ in}$ $\quad \text{let } (w, c'_\ell) = \ell.putr(y, c_\ell) \text{ in}$ $\quad \quad ((z, w), (c'_k, c'_\ell))$ $= \text{let } (x, c'_k) = k.putl(z, c_k) \text{ in}$ $\quad \text{let } (y, c'_\ell) = \ell.putl(w, c_\ell) \text{ in}$ $\quad \quad ((x, y), (c'_k, c'_\ell))$

This means, that we can represent lenses by wiring diagrams where composition corresponds to chaining and the tensor product juxtaposition



We do not know whether the category of symmetric lenses admits a *trace* in the sense of traced monoidal categories [2]. In the wiring analogy this would correspond to allowing feedback loops; algebraically, we would need to construct from a lens $\ell : X \times Y \leftrightarrow X \times Z$ a trace $\text{tr}(\ell) : Y \leftrightarrow Z$.

Graphically, this corresponds to joining the two X -ends of ℓ with a “feedback” wire. This trace operation should validate all equations that hold “graphically” in this sense.

2.3 Sums and lists

We now turn to the definition of lenses acting on disjoint union sets (“sums”) and will then use them to synchronise lists and trees using recursion.

$\frac{k \in X \leftrightarrow Z \quad \ell \in Y \leftrightarrow W}{k \oplus \ell \in X + Y \leftrightarrow Z + W}$	
C $init$ $putr(inl(x), (c_k, c_\ell))$ $putr(inr(y), (c_k, c_\ell))$ $putl(inl(z), (c_k, c_\ell))$ $putl(inr(w), (c_k, c_\ell))$	$= k.C \times \ell.C$ $= (k.init, \ell.init)$ $= \text{let } (z, c'_k) = k.putr(x, c_k) \text{ in } (inl(z), (c'_k, c_\ell))$ $= \text{let } (w, c'_\ell) = \ell.putr(y, c_\ell) \text{ in } (inr(w), (c_k, c'_\ell))$ $= \text{let } (x, c'_k) = k.putl(z, c_k) \text{ in } (inl(x), (c'_k, c_\ell))$ $= \text{let } (y, c'_\ell) = \ell.putl(w, c_\ell) \text{ in } (inr(y), (c_k, c'_\ell))$

This yields another symmetric monoidal structure. We remark that there are various ways to define an *injection lens*, but none of them commutes in the natural way with $\oplus$.

$$inl_x \in X \leftrightarrow X + Y$$

Next, we come to a list mapping lens that synchronise between two lists in a point-wise fashion using a given lens to synchronise entries. The complement maintains a mapping between the respective positions.

$\frac{\ell \in X \leftrightarrow Y}{map(\ell) \in X^* \leftrightarrow Y^*}$	
C $init$ $putr(x, c)$ $putl$	$= (\ell.C)^{\omega}$ $= (\ell.init)^{\omega}$ $= \text{let } \langle x_1, \dots, x_m \rangle = x \text{ in}$ $\quad \text{let } \langle c_1, \dots \rangle = c \text{ in}$ $\quad \text{let } (y_i, c'_i) = \ell.putr(x_i, c_i) \text{ in}$ $\quad (\langle y_1, \dots, y_m \rangle, \langle c'_1, \dots, c'_m, c_{m+1}, \dots \rangle)$ (similar)

We can also define a lens akin to **fold-left**  from functional programming provided that we have a decreasing weight function. Given $\ell : Unit + X \times Z \leftrightarrow Z$ can define $fold(\ell) : X^* \leftrightarrow Z$ such that

$$\begin{array}{ccc}
 Unit + X \times X^* & \xrightarrow{iso} & X^* \\
 \downarrow id_{Unit} \oplus (id_X \otimes fold(\ell)) & & \downarrow fold(\ell) \\
 Unit + X \times Z & \xrightarrow{\ell} & Z
 \end{array}$$

Folding can be generalised to trees and yields the following useful special cases.

- $\text{leaves} : \text{Tree } A \leftrightarrow [A]$
- $\text{concat} : [[A]] \leftrightarrow [A]$
- $\text{partition} : [A \uplus B] \leftrightarrow [A] \times [B]$
- $\text{map} : (A \leftrightarrow B) \rightarrow ([A] \leftrightarrow [B])$

Notice in particular that the mapping lens defined explicitly before can alternatively obtained as an instantiation of the fold lens just as one knows from functional programming.

We remark that the versions of sums and lists described here are called *retentive*. This means that when we change sides or extend list length we use the “retained” values from the last time we were on that side / had that length. There is also a *forgetful* version where upon shortening or changing sides we throw data away.

We also note that since lenses are self-dual one can easily define *hylomorphisms*: from $k : Z \leftrightarrow \text{Unit} + X \times Z$ and $\ell : \text{Unit} + X \times W \leftrightarrow W$ obtain $\text{Hy}(\ell, k) : Z \leftrightarrow W$ such that the obvious diagram commutes. One can also define iterators over more than one list and generalise the whole setup to other inductive datatypes.

For other non-inductive datatypes, in particular labelled graphs we can use containers as described in the next subsection.

2.4 Containers

Containers [1] are a general framework for data structures with *positions* holding *data*. They are closely related to combinatorial species [14], see also https://en.wikipedia.org/wiki/Combinatorial_species. Containers subsume inductive types like lists or trees are also containers, but not all containers are inductive types, e.g. labelled graphs are not.

Definition 4. *A container consists of*

- a set I of *shapes*
- for each shape $i \in I$ a set $B(i)$ of *positions*.

A container $T = (I, B)$ defines a functor on Sets: $T(X) = \sum_{i \in I} X^{B(i)}$. An element of $T(X)$ consists of a shape i and for each position $p \in B(i)$ an element of X .

For lists we take $I = \mathbb{N}$ and $B(i) = \{0, \dots, i-1\}$. In this case, we have $T(X) = \sum_{n \in \mathbb{N}} X^{\{0, \dots, n-1\}}$ which is isomorphic to the set of lists X^* .

In the case of graphs with X -labelled nodes we take for I the set of unlabelled graphs and for $B(i)$ the nodes of i .

For a function $f : X \rightarrow Y$ the morphism part $T(f) : T(X) \rightarrow T(Y)$ applies f to each entry of a container. It generalises “map” on lists and trees.

Our aim is to generalise $T(f)$ to $T(\ell)$ with ℓ a lens. If the shape does not change we can just apply the lens position-wise. In order to deal with shape changes, say from i to i' , we need some additional structure:

Definition 5. An ordered container is a container (I, B) with the following additional structure.

- the set of positions I forms a partial ordering (poset) with binary meets on shapes. Here, $i \leq i'$ means that i is a sub-shape of i' , e.g., sub-tree or shorter list. The meet of, say, two trees is the largest common sub-tree of the two.
- B forms a functor from the poset I to the category of sets: if $i \leq i'$ then there is an injection $B(i) \hookrightarrow B(i')$.
- If $p \in B(i)$ and $p' \in B(i')$ are equal in $B(j)$, thus, $i \leq j, i' \leq j$ then there must exist unique $q \in B(i \wedge i')$ such that p and p' arise from q by applying injections, i.e. B is a pullback preserving functor from I to the category of sets.

We are now ready to define a container mapping lens. For (I, B) an ordered container we obtain the following lens combinator.

$$\frac{\ell \in X \leftrightarrow Y}{F_{I,B}(\ell) \in F_{I,B}(X) \leftrightarrow F_{I,B}(Y)}$$

$$C = \{t \in \prod_{i \in I} B(i) \rightarrow \ell.(C) \mid \forall i, i'. i \leq i' \supset \forall b \in B(i). t(i')(b|i') = t(i)(b)\}$$

$$\begin{aligned} \text{init}(i)(b) &= \ell.\text{init} \\ \text{putr}((i, f), t) &= \\ \text{let } f'(b) &= \text{fst}(\ell.\text{putr}(f(b), t(i)(b))) \text{ in} \\ \text{let } t'(j)(b) &= \\ \text{if } \exists b_0 \in B(i \wedge j). b_0|j = b & \\ \text{then } \text{snd}(\ell.\text{putr}(f(b_0|i), t(j)(b))) & \text{ in} \\ \text{else } t(j)(b) & \\ ((i, f'), t') & \\ \text{putl} & \quad (\text{similar}) \end{aligned}$$

In [9] we also discuss a restructuring lens that can synchronise between different container types. See also Section 3.

2.5 Spans of lenses

There is a close relationship between asymmetric and symmetric lenses via spans. This was noted in [9] and elaborated in detail by Johnson et al., see [13, 12] and also this volume. that such symmetric lenses are equivalent to *spans of lenses*, i.e., two views on a common source in the sense of asymmetric lenses.

Every asymmetric lens, i.e., a classical lens in the sense of Foster et al., gives rise to a symmetric lens.

$\frac{\ell \in X \xrightarrow{\alpha} Y}{\ell^{sym} \in X \leftrightarrow Y}$	
C $init$ $putr(x, f)$ $putl(y, f)$	$= \{f \in Y \rightarrow X \mid \forall y \in Y. \ell.get(f(y)) = y\}$ $= \ell.create$ $= (\ell.get(x), f_x)$  $= \text{let } x = f(y) \text{ in } (x, f_x)$

But not all symmetric lenses are of that form, e.g., the opposite of an asymmetric lens is not an asymmetric lens unless it comes from a bijection.

However, for any lens ℓ we can find *asymmetric* lenses k_1, k_2 such that

$$(k_1^{sym})^{op}; k_2^{sym} = \ell$$

For the “intermediate type”, i.e., the common source of k_1 and k_2 we use the set of consistent triples for ℓ :

$$S_\ell = \{(x, y, c) \in X \times Y \times \ell.C \mid \ell.putr(x, c) = (y, c)\}$$

If $\ell : X \leftrightarrow Y$ then indeed $k_1 : S_\ell \rightarrow X$ and $k_2 : S_\ell \rightarrow Y$ where k_1 and k_2 are the obvious projections.

2.6 Summary

Symmetric lenses generalise the asymmetric lenses by Foster et al to truly bidirectional synchronisation. They can be seen as symmetrised asymmetric lenses with a shared complement, as stateful back-and-forth transformations, and as spans of asymmetric lenses. We have also seen that it is useful to understand symmetric lenses modulo bisimulation which coincides with observational equivalence in a suitable sense. The symmetric lenses with sets as objects form a category whose structure we have begun to explore. In particular, we identified two kinds of tensor product one akin to cartesian product and the other one akin to disjoint union of sets. We also showed how the category of lenses supports inductive and container-like datatypes and thus bidirectional synchronisation between them.

Further work and possible projects in the area of symmetric lenses include the integration with programming and implementations, a further exploration of the possibility of defining lenses by recursion and an investigation of possible higher-order structure.

3 Edit Lenses

As we have seen, both asymmetric and symmetric lenses transport entire states across the link. They do not differentiate between small changes and altogether new setups of either side of a lens. Of course, the complement can be used to

store parts or all of the current state of one or both sides of a lens and so one can work out differences using a diff-like algorithm. But, as is well-known such diffing methods invariably have to rely on heuristics and cannot, for example, distinguish between swaps and deletions followed by insertions of elements in a data structure. To remedy this, we introduce *edit languages* which are special kinds of monoids representing edits to a data structure. An *edit lens* then transports and translates elements of the edit language which are to be applied at the other end. We shall see that the ordinary symmetric lenses, henceforth called *state-based* lenses arise as a special case of edit lenses and that much of the existing category-theoretic structure carries over to the edit case. The powerful fold-combinators for inductive data types do, however, not generalise and must therefore be replaced with a selection of combinators for mapping, restructuring, and partitioning.

3.1 Edit languages

An edit language for a set X is defined as a monoid M together with a partial monoid action of M on X :

$$\begin{aligned} \mathbf{1}_M \cdot m &= m \cdot \mathbf{1}_M = m \\ m_1 \cdot (m_2 \cdot m_3) &= (m_1 \cdot m_2) \cdot m_3 \\ \mathbf{1}_M \odot x &= x \\ (m_1 \cdot m_2) \odot x &= m_1 \odot (m_2 \odot x) \end{aligned}$$

The last two equations are understood in the sense of Kleene equality: if one side is defined so is the other and they are equal. In general, however, a term $m \odot x$ may be undefined.

If we have a set X we often use the notation ∂X for an associated edit language. For example, if X is a set we may consider the following edit language for the set X^* of lists over X . We first define *atomic edits* E for X^* :

- $\text{modify}(p, dx)$ where $p \in \mathcal{N}$, $dx \in \partial X$
- $\text{resize}(i, j, x)$ where $i, j \in \mathcal{N}$, $x \in X$
- $\text{reorder}(i, f)$ where f permutes $\{0, \dots, i\}$

We then take $\partial(X^*)$ as the free monoid E^* of atomic edits. The action of atomic edits is implicit in the names of the edits, e.g. $\text{modify}(p, dx)$ succeeds on $l = [x_0, \dots, x_{n-1}]$ if $p < n$ and if $dx \odot x_p$ succeeds. The result then is

$$[x_0, \dots, x_{p-1}, dx x_p, x_{p+1}, \dots, x_{n-1}]$$

This action is then extended to lists of edits by sequential application. In general, $\mathbf{1}$ represents the neutral edit that does nothing and the product $m \odot m'$ represents the combined edit whose effect is to first apply m' and then m . Often, edits are construed as sequences of atomic edits, but this does not have to be so

for sometimes, laws may be imposed between edits, e.g., certain edits might commute with one another or cancel each other out.

This happens, for example in the “overwrite” monoid which we use in order to embed state-based into edit based lenses. For each element $x \in X$ it has an edit whose effect is to replace the current state with x . Another example is given by product monoids which we use as an edit language for cartesian products. There we take the view that edits to different components of a cartesian product commute with each other.

Following algebraic tradition we call a pair of a set and an edit language, i.e., a monoid acting partially on it, a *module*. We have already sketched how every set X can be regarded as a module. Formally, $\partial X = X \cup \{1\}$ and $x \odot x' = x$, etc.

We also explained how the cartesian product of modules is a module again.

As for sums we adopt the following approach. Primitive edits for the disjoint union $X + Y$ are

$$\begin{aligned} G_{X,Y}^{\oplus} = & \{\text{switch}_{iL}(dx) \mid i \in \{L, R\}, dx \in \partial X\} \\ & \cup \{\text{switch}_{iR}(dy) \mid i \in \{L, R\}, dy \in \partial Y\} \\ & \cup \{\text{stay}_L(dx) \mid dx \in \partial X\} \cup \{\text{stay}_R(dy) \mid dy \in \partial Y\} \\ & \cup \{\text{fail}\} \end{aligned}$$

We then define $\partial(X + Y) = (G_{X,Y}^{\oplus})^*$. Again, we omit the obvious definition of the action of primitive elements.

We tried to impose reasonable equations on $\partial(X \otimes Y)$ so as to achieve the expected isomorphism $X \oplus (Y \oplus Z) \simeq (X \oplus Y) \oplus Z$, but did not succeed in doing so.

Now reconsider the aforementioned edit language for lists. We give its primitive edits with slightly more detail.

$$\begin{aligned} G_X^{\text{list}} = & \{\text{mod}(p, dx) \mid p \in \mathbb{N}^+, dx \in \partial X\} \\ & \cup \{\text{ins}(i) \mid i \in \mathbb{N}\} \cup \{\text{del}(i) \mid i \in \mathbb{N}\} \\ & \cup \{\text{reorder}(f) \mid \forall i \in \mathbb{N}. f(i) \text{ permutes } \{1, \dots, i\}\} \\ & \cup \{\text{fail}\} \end{aligned}$$

Here, an interesting open question is to what extent such an edit language can be derived automatically from the recursive definition of lists by $X^* = 1 + X \times X^*$. Other interesting points for thought include the question why it is important to have a partial action rather than a total one and why it is reasonable to distinguish between a monoid element, i.e., an edit and the function it induces on the underlying set. The paper on edit lenses [10] contains some ideas on these questions.

3.2 Stateful homomorphisms

We now describe how to synchronise two edit languages. As in the state-based case, the translations will be history-dependent and thus require a shared complement set.

Definition 6. Given monoids M and N and a complement set C , a stateful monoid homomorphism from M to N over C is a function $h \in M \times C \rightarrow N \times C$ satisfying two laws:

$$\overline{h(\mathbf{1}_M, c)} = (\mathbf{1}_N, c)$$

$$\frac{h(m, c) = (n, c') \quad h(m', c') = (n', c'')}{h(m' \cdot_M m, c) = (n' \cdot_N n, c')}$$

It is an interesting exercise or question to reformulate this definition as an instance of a standard homomorphism between a different kind of monoids.

3.3 Edit lenses

We now come to the central definition of an edit lens. It operates between two modules and, in addition to a stateful homomorphism, comprises a consistency relation which, as we recall, was definable in the state-based setting but does not seem to be in the present case.

Definition 7. An edit lens $\ell : \langle M, X \rangle \leftrightarrow \langle N, Y \rangle$ has:

- a complement set C of private data
- consistency relation $K \in X \times C \times Y$
- stateful monoid homomorphisms

$$\Rightarrow : M \times C \rightarrow N \times C$$

$$\Leftarrow : N \times C \rightarrow M \times C$$

that preserve consistency

in the sense that if $(x, c, y) \in K$ and $\Rightarrow(dx, c) = (dy, c')$ then $(dx, x, c', dy, y) \in K$ and symmetrically for $\Leftarrow$. Again, we use dot notation to refer to the components of an edit lens.

As before, we may consider lenses up to equivalence thus obtaining a category.

Definition 8 (Lens equivalence). Two lenses $k, \ell : X \leftrightarrow Y$ are equivalent (written $k \equiv \ell$) if, there exists a relation $S \subseteq X \times k.C \times \ell.C \times Y$ such that

- $(\text{init}_X, k.\text{init}, \ell.\text{init}, \text{init}_Y) \in S$;
- if $(x, c, d, y) \in S$ and $dx \cdot x$ is defined, then if $(dy_1, c') = k.\Rightarrow(dx, c)$ and $(dy_2, d') = \ell.\Rightarrow(dx, d)$, then $dy_1 = dy_2$ and $(dx, x, c', d', dy_1, y) \in S$; and
- analogously for $\Leftarrow$.

Again, there is an equivalent definition with dialogues whose precise definition we omit. We now give the definitions of product and sum lenses.

$$\frac{k \in X \leftrightarrow Z \quad \ell \in Y \leftrightarrow W}{k \otimes \ell \in X \otimes Y \leftrightarrow Z \otimes W}$$

$$\begin{aligned} C &= k.C \times \ell.C \\ \text{init} &= (k.\text{init}, \ell.\text{init}) \\ K &= \{ ((x, z), (c_k, c_\ell), (y, w)) \mid \\ &\quad (x, c_k, y) \in k.K \\ &\quad \wedge (z, c_\ell, w) \in \ell.K \} \end{aligned}$$



$$\frac{k \in X \leftrightarrow Y \quad \ell \in Z \leftrightarrow W}{k \oplus \ell \in X \oplus Z \leftrightarrow Y \oplus W}$$

$$\begin{aligned} C &= k.C + \ell.C \\ \text{init} &= \text{inl}(k.\text{init}) \\ K &= \{ (\text{inl}(x), \text{inl}(c), \text{inl}(y)) \mid (x, c, y) \in k.K \} \\ &\quad \cup \{ (\text{inr}(z), \text{inr}(c), \text{inr}(w)) \mid (z, c, w) \in \ell.K \} \\ c_k &= k.\text{init} \\ c_\ell &= \ell.\text{init} \\ \Rightarrow_g(\text{switch}_{LL}(\text{dx}), \text{inl}(c)) &= \text{let } (\text{dy}, c') = k.\Rightarrow(\text{dx}, c_k) \\ &\quad \text{in } (\text{switch}_{LL}(\text{dy}), \text{inl}(c')) \\ \Rightarrow_g(\text{switch}_{RL}(\text{dx}), \text{inr}(c)) &= \text{let } (\text{dy}, c') = k.\Rightarrow(\text{dx}, c_k) \\ &\quad \text{in } (\text{switch}_{RL}(\text{dy}), \text{inl}(c')) \\ \Rightarrow_g(\text{switch}_{LR}(\text{dz}), \text{inl}(c)) &= \text{let } (\text{dw}, c') = \ell.\Rightarrow(\text{dz}, c_\ell) \\ &\quad \text{in } (\text{switch}_{LR}(\text{dw}), \text{inr}(c')) \\ \Rightarrow_g(\text{switch}_{RR}(\text{dz}), \text{inr}(c)) &= \text{let } (\text{dw}, c') = \ell.\Rightarrow(\text{dz}, c_\ell) \\ &\quad \text{in } (\text{switch}_{RR}(\text{dw}), \text{inr}(c')) \\ \Rightarrow_g(\text{stay}_L(\text{dx}), \text{inl}(c)) &= \text{let } (\text{dy}, c') = k.\Rightarrow(\text{dx}, c) \\ &\quad \text{in } (\text{stay}_L(\text{dy}), \text{inl}(c')) \\ \Rightarrow_g(\text{stay}_R(\text{dz}), \text{inr}(c)) &= \text{let } (\text{dw}, c') = \ell.\Rightarrow(\text{dz}, c) \\ &\quad \text{in } (\text{stay}_R(\text{dw}), \text{inr}(c')) \\ \Rightarrow_g(e, c) &= (\text{fail}, c) \text{ in all other cases} \\ \Leftarrow_g &\text{ is analogous} \end{aligned}$$

As already mentioned a list mapping *edit* lens can, as far as we know, be construed from a more general fold-pattern so we give it here as a primitive combinator.

$\frac{\ell \in X \leftrightarrow Y}{\ell^* \in X^* \leftrightarrow Y^*}$	
C	$= \ell.C^*$
$init$	$= \varepsilon$
K	$= \{(x, c, y) \mid x = c = y \wedge \forall 1 \leq p \leq x . (x_p, c_p, y_p) \in \ell.K\}$
$\Rightarrow_g(\text{mod}(p, dx), c)$	$= \text{let } (dy, c'_p) = \ell.\Rightarrow(dx, c_p) \text{ in } (\text{mod}(p, dy), c[p \mapsto c'_p])$ when $p \leq n$
$\Rightarrow_g(\text{mod}(p, dx), c)$	$= (\text{fail}, c) \text{ when } p > n$
$\Rightarrow_g(\text{fail}, c)$	$= (\text{fail}, c)$
$\Rightarrow_g(dx, c)$	$= (dx, dx\ c) \text{ in all other cases}$
$\Leftarrow$	similar

As a concrete application we repeat in Figure 2 a running example from [10] which illustrates the lenses we have seen so far.

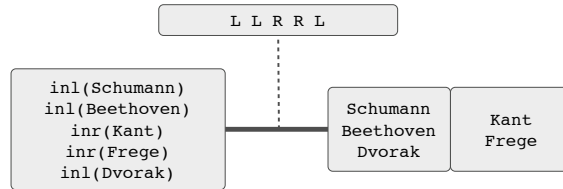
3.4 The partition lens

We seek a lens of the form

$$partition \in (X \oplus Y)^* \leftrightarrow X^* \otimes Y^*$$

...

Once we have it, we can compose many important lenses on lists from it: e.g., we can use mapping to go from Z^* to $(X + Y)^*$ and then on using partition and we can further process $X^* \times Y^*$ by working on both parts separately using the tensor lens after which we can go back to a single list type W^* using the opposite of partitioning. Figure 3 (taken from [10]) contains the code for the partition lens. From a bird's eye perspective we recognise the complement $C = \{L, R\}^*$ which tells where the positions of the LHS belong. The consistent triples can thus be visualised as in



The example in Figure 4 (where L, R are written `inl`, `inr`) gives an idea how edits are propagated. We refer to loc.cit. for details.



(a) initial replicas

`ins(3);`
`mod(3, ("Monteverdi", "1567-1643"))`



(b) a new composer is added to one replica

`ins(3);`
`mod(3, ("Monteverdi", 1))`



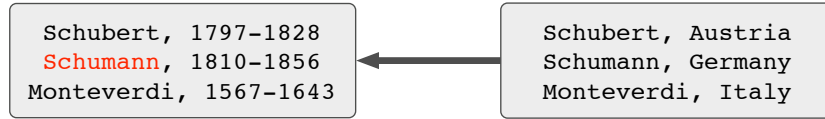
(c) the lens adds the new composer to the other replica

`mod(3, (1, "Italy"));`
`mod(2, ("Schumann", 1))`



(d) the curator makes some corrections

`1;`
`mod(2, ("Schumann", 1))`



(e) the lens transports a small edit

`del(3); ins(1);`
`mod(1, ("Monteverdi", "1567-1643"))`

`del(3); ins(1);`
`mod(1, ("Monteverdi", 1))`



`reorder(3,1,2)`

`reorder(3,1,2)`

(f) two different edits with the same effect on the left

Fig. 2. Synchronising composers



partition $\in (X \oplus Y)^* \leftrightarrow X^* \oplus Y^*$	
C	$= \{L, R\}^*$
out	$= \{z \in \text{map}_{\text{map}}(c), (\text{left}(c), \text{right}(c)) \mid z \in (X \oplus Y)^*\}$
K	$= \{z \in \text{map}_{\text{map}}(c), (\text{left}(c), \text{right}(c)) \mid z \in (X \oplus Y)^*\}$
$\varphi_1(\text{mod}(p, dx), c)$	$= \{\text{fail}, c\}$ when $p > c $ (1)
$\varphi_2(\text{mod}(p, c), c)$	$= \{c, c\}$ when $1 \leq p \leq c $ (2)
$\varphi_3(\text{mod}(p, dx \oplus c), c)$	$= \{d, d, c'\}$ where $1 \leq p \leq n$ (3)
$\varphi_4(\text{mod}(p, \text{switch}_k(dx)), c)$	$= \{d, dx, dx, c, p \mapsto k\}$, where $(p_1, p_2) = \text{count}(p, c)$ (4)
$\varphi_5(\text{mod}(p, \text{stay}(dx)), c)$	$= \{\text{tag}(j, \text{mod}(p, dx)), c\}$, where $(p_1, p_2) = \text{count}(p, c)$ (5)
$\varphi_6(\text{mod}(p, \text{fail}), c)$	$= \{\text{fail}, c\}$ (6)
$\varphi_7(\text{ins}(i), c)$	$= \{\text{left}(\text{ins}(i)), \text{ins}(i), c\}$ (7)
$\varphi_8(\text{del}(i), c)$	$= \{d, dx, \text{del}(i), c\}$, where $(n_1, n_2) = \text{count}(i+1, c')$ (8)
$\varphi_9(\text{reorder}(f), c)$	$= \{d, dx, c'\}$, where $(n_1, n_2) = \text{count}(i+1, c')$ (9)
$\varphi_{10}(\text{fail}, c)$	$= \{\text{fail}, c\}$ (10)
$\varphi_{11}(c, c)$	$= \{c, c\}$ (11)
$\varphi_{12}(\text{div}(n), c)$	$= \{d, d, c'\}$ when $n > 1$, where $(d, c') = \varphi_{12}(\text{div}(n), c)$ (12)
$\varphi_{13}(\text{left}(\text{mod}(p, dx)), c)$	$= \{\text{stay}_1(\text{mod}(p, dx)), c\}$, where $p' = \text{ins}(c)^{-1}(\text{mod}(p))$ (13)
$\varphi_{14}(\text{left}(\text{reorder}(f)), c)$	$= \{\text{reorder}(f), c\}$, where $p(\text{ins}(p)) = \text{ins}(p)$ (14)
$\varphi_{15}(\text{left}(\text{ins}(i)), c)$	$= \{\text{ins}(i), \text{ins}(i), c\}$ (15)
$\varphi_{16}(\text{left}(\text{del}(i)), c)$	$= \{c, c\}$ (16)
$\varphi_{17}(\text{left}(\text{del}(i)), c)$	$= \{d, d, c'\}$, where $(n_1, n_2) = \text{count}(i, c)$ (17)
$\varphi_{18}(\text{left}(\text{del}(i)), c)$	$= \{c, c\}$ when $i < i \leq n_2 + 1$ (18)
$\varphi_{19}(\text{left}(\text{fail}), c)$	$= \{\text{fail}, c\}$ (19)
$\varphi_{20}(\text{right}(dx), c)$	$= \{\text{fail}, c\}$ (20)

Fig. 3. Partition lens, the code view.

3.5 Containers

As in the state-based case we now show how to synchronise between container types using edit lenses. For technical reasons we need a slightly different formulation of containers, the difference being on the one hand that shapes and positions must be editable, i.e., modules, and on the other hand that positions are given as a single set rather than a family of sets which facilitates the definition of rearrangements.

Definition 9. An editable container, henceforth “container”, is given by the following data:

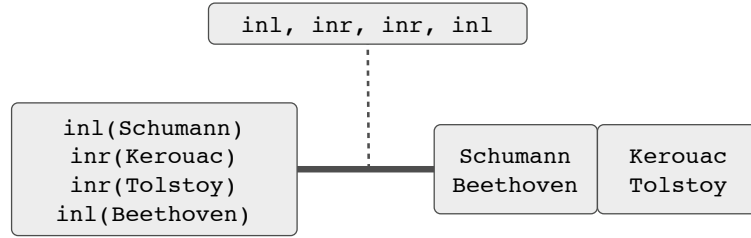
- A module I of shapes additionally endowed with a partial order,
- A fixed set P of positions
- For each shape i a subset $\text{live}(i) \subseteq P$.

We notice that we can recover the traditional formulation by $B(i) = \{p \mid p \in \text{live}(i)\}$ and as before, a container type $T = \langle I, P, \text{live} \rangle$ defines a type operator by $T(X) = \sum_{i \in I} \text{live}(i)$.

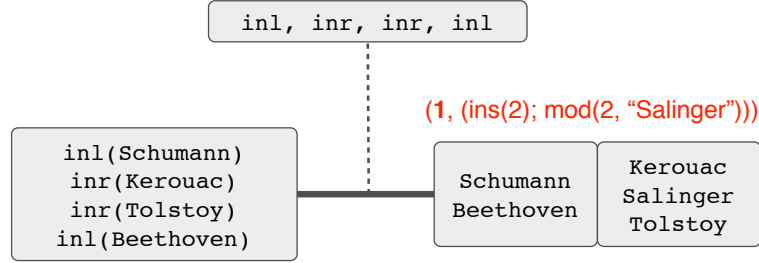
Let $T = \langle I, P, \text{live} \rangle$ be a container type. An edit $di \in \partial I$ is an *insertion* if $di \ i \geq i$ whenever defined. It is a *deletion* if $di \ i \leq i$ whenever defined. It is a *rearrangement* if $|\text{live}(di \ i)| = |\text{live}(i)|$ (same cardinality) whenever defined. We only employ edits from these three categories as ingredients of container edits; any other edits in the module will remain unused. This division of container edits into “pure” insertions, deletions, and rearrangements facilitates the later definition of lenses operating on such edits.

Definition 10. We define the monoid of edits for a container type $\langle I, P, \text{live} \rangle$ as the free monoid generated by

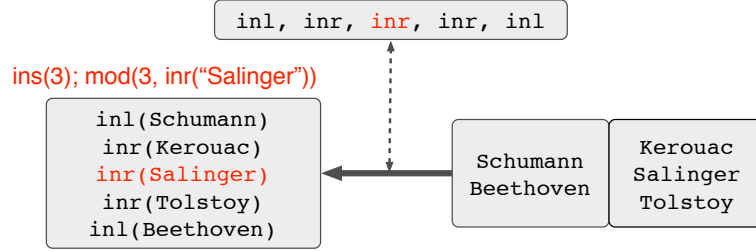
- Modifications: $\text{mod}(p, dx)$ where $p \in P$ and $dx \in \partial X$,
- Insertions: $\text{ins}(di)$ with di an insertion,



(a) the initial replicas: a tagged list of composers and authors on the left; a pair of lists on the right; a complement storing just the tags



(b) an element is added to one of the partitions



(c) the complement tells how to translate the index

Fig. 4. Propagating edits across the partition lens

$$\begin{array}{c}
\begin{array}{c}
T = \langle I, P, \text{live} \rangle \text{ a container type} \\
T' = \langle I', P', \text{live}' \rangle \text{ a container type} \\
\ell \in I \leftrightarrow I' \\
\hline
[T, T'](\ell) \in T(X) \leftrightarrow T'(X)
\end{array} \\
\\
\begin{array}{lcl}
C & = & \ell.K \\
\text{init} & = & (\text{init}_I, \ell.\text{init}, \text{init}_{I'}) \\
K = \{((i, f), (i, c, i'), (i', f')) \\
& \mid (i, c, i') \in \ell.K \wedge \forall p \in \text{live}'(i'). f(f_{i,c,i'}(p)) = f'(p)\} \\
\Rightarrow_g(\text{mod}(p, dx), (i, c, i')) & = & (\text{mod}(f_{i,c,i'}^{-1}(p), dx), (i, c, i')) \\
& & \text{when } p \in \text{live}(i) \\
\Rightarrow_g(\text{ins}(di), (i, c, i')) & = & (\text{rearr}(\mathbf{1}, f_i)\text{ins}(di'), \\
& & (di \ i, c', di' \ i')) \\
\Rightarrow_g(\text{del}(di), (i, c, i')) & = & (\text{rearr}(\mathbf{1}, f_d)\text{del}(di'), \\
& & (di \ i, c', di' \ i')) \\
\Rightarrow_g(\text{rearr}(di, f), (i, c, i')) & = & (\text{rearr}(di', f_r), \\
& & (di \ i, c', di' \ i'))
\end{array}
\end{array}$$

in the last three clauses: $(di', c') = \ell.\Rightarrow(di, c)$

Three families of bijections f_i, f_d, f_r must be chosen in such a way that the container edits in which they appear are well-formed (this is possible since di' is an insertion, deletion, or rearrangement as appropriate) and such that the following three constraints are satisfied: in each case i, i' , etc., refer to the current values from above and $p \in \text{live}'(di' \ i')$ is an arbitrary position.

$$\begin{aligned}
f_i(di' \ i')(p) &= f_{i,c,i'}^{-1}(f_{di \ i, c', di' \ i'}(p)) \\
&\quad \text{when } f_{di \ i, c', di' \ i'}(p) \in \text{live}(i) \\
f_d(di' \ i')(p) &= f_{i,c,i'}^{-1}(f_{di \ i, c', di' \ i'}(p)) \\
f_r(di' \ i')(p) &= f_{i,c,i'}^{-1}(f(i)(f_{di \ i, c', di' \ i'}(p)))
\end{aligned}$$

As an example of container restructuring we demonstrate in Figure 20 a lens for in-order flattening of a tree.

3.6 Typed edit language

We have experimented with *typed edit languages* which should (tentatively!) comprise

- a set T of “types”
- for each $t \in T$ a set $X(t)$ with distinguished element $\text{init}_{X(t)} \in X(t)$
- for any two types t, t' a set of edits $\partial X(t, t')$ with composition and identities, i.e. a category!
- an action of ∂X on X : if $e \in \partial X(t, t')$ and $x \in X(t)$ then $e.x \in X(t')$. I.e. $X(-)$ becomes a set-valued functor (presheaf).

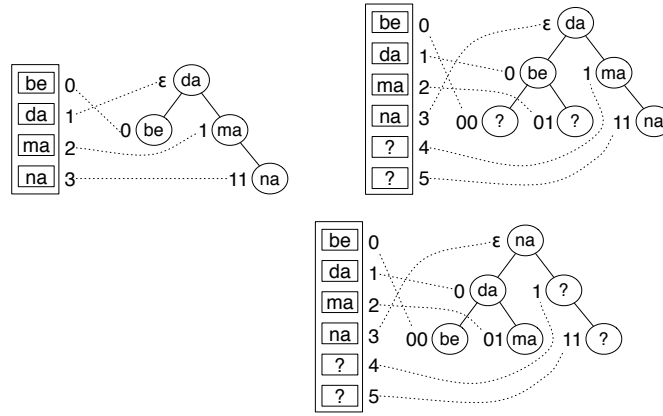


Fig. 5. Inserting two fresh nodes at the end of the list, propagation and restoration of consistency.

This would allow one to distinguish lists by their lengths and thus to avoid border cases with head and tail of the empty list. It might also provide a solution to the problem that disjoint union for edit lenses is not associative by distinguishing the two components of a sum with types. There is also a relationship with the modelling of edits as categories in, for example, Johnson’s work [13, 12], this volume. There, however, the states, i.e., editable objects themselves, form the object of the category and not the types. As a result, an edit can only be applied to a single object (its domain in the category-theoretic sense). We prefer the viewpoint that an edit is an operation that as such can be applied to different objects. In the case of modules these objects are all elements of the underlying set, in the typed case these are the objects of the domain type of the edit.

Further exploration of the idea of typed edit languages is left as an open problem.

3.7 State-based to edit-based and back

In this section we explain how to relate state-based lenses with edit based ones. Let X be a set. The free monoid X^* acts on X by

$$(x_n \dots x_1)x = x_n$$

For $x \in X$ define module X_x as $X_x = (X, x, X^*)$. 

Let $\ell : X \leftrightarrow Y$ be a *state-based* symmetric lens and $\ell.putr(x, \ell.missing) = (y, \ell.missing)$ be a consistent triple for ℓ .

It is now an easy exercise to define an edit based lens between X_y and Y_y .

Conversely, if X is a module let a *differ* for X be a binary operation $dif \in X \times X \rightarrow \partial X$ satisfying $dif(x, x')x = x'$ and $dif(x, x) = \mathbf{1}$. Thus, a differ finds, for given states x, x' , an edit operation dx such that $dx \ x = x'$ and dx

is “reasonable” at least in the sense that if $x = x'$ then the produced edit is minimal, namely **1**.

It is instructive to ponder possible differs for the module X^* , say for trivial X .

Now, given an edit-based lens $\ell : X \leftrightarrow Y$ and differs for X and Y we obtain a state-based lens by

$\overline{k : X \leftrightarrow Y}$	
C $init$ $putr(x', (x, c, y))$	$= \ell.K$ $= (init_X, \ell.init, init_Y)$ $= (dx\ x, c', dy\ y)$ where $dx = dif(x, x')$ and $(dy, c') = \ell.\Rightarrow(dx, c)$

4 Information trees & Conclusion

Before concluding this section outlines an approach for edit lens primitives for trees and (later on) graphs with unordered children as in XML or web applications. Various options beyond hand-crafting from the definitions could be containers modulo reordering of children positions. These, in fact, are the aforementioned combinatorial species which in this sense strictly generalise the containers.

Here, instead, we define unordered trees from scratch and then use tree automata to describe well-formed subsets. Using weakest preconditions we are then able to characterise those edit operations which preserve well-formedness and in this way obtain natural instance of typed edit languages.

Defining lenses on top of these edit languages is left for future work here.

4.1 Information trees

An *information tree* is a finite tree with unordered children whose *edges* are labelled with words over a fixed finite alphabet Σ . Using linear notation with set braces $\{\}$ and arrows $\mapsto$ we have the following example

$$\{\text{name} \mapsto \{\text{John} \mapsto \{\}\}\}, \text{email} \mapsto \{\text{john@example.com} \mapsto \{\}\}$$

or in abbreviated form: same in abbreviated form:

$$\{\text{name} \mapsto \text{John}, \text{email} \mapsto \text{john@example.com}\}$$

We consider the following primitive edits.

$$\begin{aligned}
 e ::= & \text{insert}(t) \mid \\
 & \text{hoist}(m, n) \mid \\
 & \text{delete}(m) \mid \\
 & \text{rename}(m, n) \mid \\
 & \text{at}(n, e)
 \end{aligned}$$

where m, n are names, and t is a tree. Here, $insert(t)$ inserts t at the root (assuming that there are no name clashes); $hoist(m, n)$ removes the n -children from the m -labelled sub-trees of the root and adds (“hoists”) them to the root; $at(n, e)$ applies e to the n -labelled children of the root and thus, allows the application of edits at arbitrary depths.

4.2 Sheaves automata

We will specify tree types (document types) by a special kind of automata, namely the sheaves automata from [4, 8].

Intuitively, a sheaves automaton has a set of states Q and for each $q \in Q$ a *sheaves formula* which partitions the allowed sub-trees into disjoint classes (recursively using states) and specifies an arithmetic constraint between the numbers of sub-trees falling into each class.

E.g. two states “person”, “address”. A “person” has one “address” labelled **address** and many “persons” labelled **friend**. An “address” has sub-trees labelled **Street**, **Town**, etc. some of them optional.

Another example of a tree type definable with a sheaves automaton is a type FS of *file systems*:

$$\begin{aligned} \text{FS} &::= (.^* \rightarrow \text{F} \mid \text{D})^* \\ \text{F} &::= \mathbf{f} \rightarrow .^* \\ \text{D} &::= \mathbf{d} \rightarrow \text{FS} \end{aligned}$$

Special naming conventions, file names starting with dot or ending with bin, etc., can also be accommodated. Further examples include tree-structured representation of program text and tree representation of game states (SGF).

It is known [4] that inclusion and nonemptiness of sheaves automata is decidable and that boolean operations are computable. Furthermore, sheaves automata have been presented as a type system [8] with subtyping where the algorithms for inclusion etc are used in order to do automatic type checking for functional programs (in a restricted syntax) producing trees.

Our result [11] asserts that for sheaves automaton A and tree edit e one can effectively compute a sheaves automaton $e.A$ such that

$$t \in L(e.A) \iff e.t \text{ fails} \vee e.t \in L(A)$$

Writing $e : A \rightarrow B$ to mean that $\forall t \in L(A). e.t \text{ defined} \Rightarrow e.t \in L(B)$, we have

$$e : A \rightarrow B \iff L(A) \subseteq L(e.B)$$

where the latter condition is decidable by the known results (as usual $L(-)$ stands for the set of accepted trees).

To see the equivalence we reason as follows: Suppose $e : A \rightarrow B$ and $t \in L(A)$. If $e.t$ is undefined then $e.t \in L(e.B)$ by definition of $e.B$. So, assume $e.t$ defined. By assumption $e.t \in L(B)$ and, again by definition of $e.B$, we have $t \in L(e.B)$.

For the converse, suppose $L(A) \subseteq L(e.B)$ and $t \in L(A)$ and $e.t$ defined. Then, $t \in L(e.B)$ and, since $e.t$ defined, $e.t \in L(B)$.

The construction of $e.B$ proceeds along the following lines.

- For every edit e define (by induction on e) a sheaves automaton D_e such that $L(D_e) = \{t \mid e.t \text{ undefined}\}$.
- For every edit e define (by induction on e) a sheaves automaton $e \star B$ such that whenever $e.t$ is defined then $t \in L(e \star B) \iff e.t \in L(B)$ (by anticipating the action of e). If $e.t$ is undefined then t may or may not be in $e \star B$.
- Then put $e.B = D_e \vee e \star B$ with $\vee$ denoting union construction for sheaves automata.

We remark that the union requires a product construction leading to a quadratic increase of the number of states and hence to exponential blowup upon nesting. It is plausible that this could be remedied by moving to a non-deterministic version of the sheaves automata.

For a concrete example, we consider the construction of $e.B$ when $e = \text{insert}(t')$: recall that this edit inserts t' at the root assuming the top-level labels of t' are not present. Thus, D_e checks that one of the top-level labels of t' is present (cardinality ≥ 1). To construct $e \star B$ we then add a new initial state s'_0 . We then label s'_0 just like s_0 (initial state of B but “as if t' is present”. E.g. if t' has an a label and s has an expression matching a then we replace the count variable x by $x + 1$. This also shows the need for actual arithmetic constraints.

As usual, we define edits as lists of primitive edits and extend the notation $e : A \rightarrow B$ to edits. We notice that by iterating the calculation of weakest preconditions this latter judgement is also decidable.

4.3 Edit languages for information trees

If A is a sheaves automaton we define an edit language A' with underlying set $L(A)$ and $\partial A' = \{e \mid e : A \rightarrow A\}$. We can automatically check whether $e \in \partial A'$ for a given edit e . This lends itself naturally to a typed generalisation where types are sheaves automata or a finite subset thereof and $\partial(A, B) = \{e \mid e : A \rightarrow B\}$.

4.4 Conclusion and next steps

We have shown the first steps towards editing and synchronising unordered trees defined by tree automata. Our approach integrates smoothly with existing edit lenses framework and combinators. In this setting the typed edit lenses considered earlier occur naturally and can be seen as synthesis with sd/delta lenses by Diskin et al [6], which replace edit languages by categories. There are several open ends that could lead to small research projects or even PhD topics. In particular, these are

- to further investigate categorical structure of lenses and edit lenses;
- to explore equations and optimisation, e.g., “deforestation”;
- to further develop lenses based on information trees;
- to study connections with logic, e.g. whether specifications can be transported across a lens;
- to make connections to recent work about weak consistency models, e.g. [3] where consistency is not always restored in full but up to certain levels in order to save bandwidth.

References

1. Michael Abbott, Thorsten Altenkirch, and Neil Ghani. Containers: constructing strictly positive types. *Theor. Comput. Sci.*, 342(1):3–27, 2005.
2. Dominic Verity André Joyal, Ross Street. Traced monoidal categories. *Mathematical Proceedings of the Cambridge Philosophical Society*, 3:447–468, 1996. see also https://en.wikipedia.org/wiki/Traced_monoidal_category.
3. Valter Balegas, Cheng Li, Mahsa Najafzadeh, Daniel Porto, Allen Clement, Sérgio Duarte, Carla Ferreira, Johannes Gehrke, João Leitão, Nuno M. Preguiça, Rodrigo Rodrigues, Marc Shapiro, and Viktor Vafeiadis. Geo-replication: Fast if possible, consistent if necessary. *IEEE Data Eng. Bull.*, 39(1):81–92, 2016.
4. Silvano Dal-Zilio and Denis Lugiez. XML schema, tree logic and sheaves automata. In Robert Nieuwenhuis, editor, *Rewriting Techniques and Applications, 14th International Conference, RTA 2003, Valencia, Spain, June 9-11, 2003, Proceedings*, volume 2706 of *Lecture Notes in Computer Science*, pages 246–263. Springer, 2003.
5. Zinovy Diskin. Algebraic models for bidirectional model synchronization. In Krzysztof Czarnecki, Ileana Ober, Jean-Michel Bruel, Axel Uhl, and Markus Völter, editors, *MoDELS*, volume 5301 of *Lecture Notes in Computer Science*, pages 21–36. Springer, 2008.
6. Zinovy Diskin, Yingfei Xiong, Krzysztof Czarnecki, Hartmut Ehrig, Frank Hermann, and Fernando Orejas. From state- to delta-based bidirectional model transformations: The symmetric case. Technical Report GSDLAB-TR 2011-05-03, University of Waterloo, May 2011.
7. J. Nathan Foster, Michael B. Greenwald, Jonathan T. Moore, Benjamin C. Pierce, and Alan Schmitt. Combinators for bi-directional tree transformations: A linguistic approach to the view update problem. *ACM Transactions on Programming Languages and Systems*, 29(3):17, May 2007. Extended abstract in *Principles of Programming Languages (POPL)*, 2005.
8. J. Nathan Foster, Benjamin C. Pierce, and Alan Schmitt. A logic your typechecker can count on: Unordered tree types in practice. In *PLAN-X 2007, Programming Language Technologies for XML, An ACM SIGPLAN Workshop colocated with POPL 2007, Nice, France, January 20, 2007*, pages 80–90, 2007.
9. Martin Hofmann, Benjamin C. Pierce, and Daniel Wagner. Symmetric lenses. In *ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, Austin, Texas, January 2011. Full version to appear in the Journal of the ACM.
10. Martin Hofmann, Benjamin C. Pierce, and Daniel Wagner. Edit lenses. In John Field and Michael Hicks, editors, *Proceedings of the 39th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2012, Philadelphia, Pennsylvania, USA, January 22-28, 2012*, pages 495–508. ACM, 2012.
11. Martin Hofmann, Benjamin C. Pierce, and Daniel Wagner. Edit languages for information trees. *ECEASST*, 57, 2013.
12. Michael Johnson, Robert Rosebrugh, and Richard Wood. Lenses, fibrations, and universal translations. *Mathematical Structures in Computer Science*, 22:25–42, 2012.
13. Michael Johnson, Robert D. Rosebrugh, and Richard Wood. Algebras and Update Strategies. *Journal of Universal Computer Science*, 16:729–748, 2010.
14. André Joyal. Une théorie combinatoire des séries formelles. *Advances in Mathematics*, 42:1–82, 1981. see also https://en.wikipedia.org/wiki/Combinatorial_species.

15. Hsiang-Shang Ko, Tao Zan, and Zhenjiang Hu. Bigul: a formally verified core language for putback-based bidirectional programming. In Martin Erwig and Tiark Rompf, editors, *Proceedings of the 2016 ACM SIGPLAN Workshop on Partial Evaluation and Program Manipulation, PEPM 2016, St. Petersburg, FL, USA, January 20 - 22, 2016*, pages 61–72. ACM, 2016.
16. Kazutaka Matsuda and Meng Wang. Applicative bidirectional programming with lenses. In Kathleen Fisher and John H. Reppy, editors, *Proceedings of the 20th ACM SIGPLAN International Conference on Functional Programming, ICFP 2015, Vancouver, BC, Canada, September 1-3, 2015*, pages 62–74. ACM, 2015.
17. Lambert Meertens. Designing constraint maintainers for user interaction, 1998. Manuscript.
18. Perdita Stevens. Bidirectional model transformations in QVT: Semantic issues and open questions. In *International Conference on Model Driven Engineering Languages and Systems (MoDELS), Nashville, TN*, volume 4735 of *Lecture Notes in Computer Science*, pages 1–15. Springer-Verlag, 2007.
19. Janis Voigtländer. Bidirectionalization for free! (pearl). In Zhong Shao and Benjamin C. Pierce, editors, *Proceedings of the 36th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2009, Savannah, GA, USA, January 21-23, 2009*, pages 165–176. ACM, 2009.
20. Janis Voigtländer, Zhenjiang Hu, Kazutaka Matsuda, and Meng Wang. Combining syntactic and semantic bidirectionalization. In Paul Hudak and Stephanie Weirich, editors, *ICFP*, pages 181–192. ACM, 2010.
21. Daniel Wagner. *Symmetric Edit Lenses: A New Foundation for Bidirectional Languages*. PhD thesis, University of Pennsylvania, 2014.
22. Yingfei Xiong, Hui Song, Zhenjiang Hu, and Masato Takeichi. Supporting parallel updates with bidirectional model transformations. In Richard F. Paige, editor, *Theory and Practice of Model Transformations, Second International Conference, ICMT 2009, Zurich, Switzerland, June 29-30, 2009. Proceedings*, volume 5563 of *Lecture Notes in Computer Science*, pages 213–228. Springer, 2009.